

Documentação da Arquitetura (Processador 18 bits)

1. Formatos de Instrução

A arquitetura utiliza uma palavra de instrução fixa de **18 bits** (bits [17:0]), dividida em três formatos principais: R, I e J.

Formato R (Tipo Registrador)

Utilizado para operações lógicas e aritméticas entre registradores.

Campo	Bits	Tamanho	Descrição
opcode	[17:15]	3 bits	Código da operação principal
rs	[14:12]	3 bits	Endereço do registrador fonte 1
rt	[11:9]	3 bits	Endereço do registrador fonte 2
rd	[8:6]	3 bits	Endereço do registrador de destino
shamt	[5:3]	3 bits	Quantidade de deslocamento (<i>Shift amount</i>)
funct	[2:0]	3 bits	Código da função específica

Formato I (Tipo Imediato)

Utilizado para instruções que carregam valores constantes, acesso à memória e desvios condicionais.

Campo	Bits	Tamanho	Descrição
opcode	[17:15]	3 bits	Código da operação principal
rs	[14:12]	3 bits	Endereço do registrador base/fonte
rt	[11:9]	3 bits	Endereço do registrador destino/comparação
x	[8]	1 bit	<i>Don't care</i> (Bit ignorado)
address/imm	[7:0]	8 bits	Valor imediato ou endereço

Formato J (Tipo Salto)

Utilizado exclusivamente para desvios incondicionais (Jump).

Campo	Bits	Tamanho	Descrição
opcode	[17:15]	3 bits	Código da operação principal
x	[14:8]	7 bits	<i>Don't care</i> (Bits ignorados)
address	[7:0]	8 bits	Endereço de destino do salto

Code Table Controller

Instrução	opcode / funct	assembly	Descrição	Formato
Soma	000 / 000	add rd, rs, rt	$rd = rs + rt$	R
Subtração	000 / 001	sub rd, rs, rt	$rd = rs - rt$	R
Multiplicação	000 / 010	mult rd, rs, rt	$rd = rs * rt$	R
Divisão	000 / 011	div rd, rs, rt	$rd = rs / rt$	R
Negação	000 / 100	not rd, rs	$rd = \neg rs$	R
Menor que	000 / 101	slt rd, rs, rt	se $rs < rt$, $rd = 1$	R
Deslocamento à esquerda	000 / 110	sll rd, rs, N	$rd = rs \ll N$	R
Deslocamento à direita	000 / 111	srl rd, rs, N	$rd = rs \gg N$	R
Carregar dados da memória	001 / xxx	lw rt, rs(N)	$rt = \text{Memória}[rs + N]$	I
Salvar dados na memória	010 / xxx	sw rt, rs(N)	$\text{Memória}[rs + N] = rt$	I
Desvio condicional	011 / xxx	beq rs, rt, ENDR	se $rs == rt$, $PC = ENDR$	I
Soma Imediata	100 / xxx	addI rt, rs, N	$rt = rs + N$	I
Desvio incondicional	111 / xxx	jmp ENDR	$PC = ENDR$	J

Assembly Structure

```

add rd, rs, rt
sub rd, rs, rt
mult rd, rs, rt
div rd, rs, rt
not rd, rs
slt rd, rs, rt
sll rd, rs, N
srl rd, rs, N
lw rd, rs(N)
sw rd, rs(N)
beq rd, rs, ENDR
addi rd, rs, N
jmp ENDR

```

Exemplo de codificação:

Assembly	Binário	Hexadecimal
add R0, R1, R2	00 0001 0100 0000 0000	01400
addI R6, R5, 7	10 0101 1100 0000 0111	25c07
lw R3, R4(20)	00 1100 0110 0001 0100	0c614
sub R1, R1, R2	00 0001 0100 0100 0001	01441

Exemplo de Programa ReMember

addl R1, R0, 5 # R1 = 5

addl R2, R0, 10 # R2 = 10

LOOP:

add R3, R1, R2 # R3 = R1 + R2

sub R2, R2, R1 # R2 = R2 - 5

beq R2, R0, FIM # Se R2 == 0, vai para FIM

jmp LOOP # Pula para LOOP

FIM:

sw R3, 0(R0) # Salva R3 na memória

Hexadecimal:

0x20205	0x2040a	0x14c0	0x2281	0x1a006	0x38002	0x10600
---------	---------	--------	--------	---------	---------	---------

Exemplo de compilador:

```
import sys
import re

# --- CONFIGURAÇÃO DA ARQUITETURA ---

# Mapeamento de Registradores
REGISTERS = {f'R{i}': i for i in range(8)} # R0 a R7

# Tabela de Instruções
# Formato: 'mnemonico': {'type': 'R/I/J', 'opcode': int, 'funct': int (opcional)}
INSTRUCTIONS = {
    # Formato R (Opcode 000)
    'add': {'type': 'R', 'opcode': 0b000, 'funct': 0b000},
    'sub': {'type': 'R', 'opcode': 0b000, 'funct': 0b001},
    'mult': {'type': 'R', 'opcode': 0b000, 'funct': 0b010},
    'div': {'type': 'R', 'opcode': 0b000, 'funct': 0b011},
    'not': {'type': 'R', 'opcode': 0b000, 'funct': 0b100}, # not rd, rs
    'slt': {'type': 'R', 'opcode': 0b000, 'funct': 0b101},
    'sll': {'type': 'R', 'opcode': 0b000, 'funct': 0b110}, # usa shamt
    'srl': {'type': 'R', 'opcode': 0b000, 'funct': 0b111}, # usa shamt

    # Formato I
```

```

'lw':    {'type': 'I', 'opcode': 0b001}, # lw rt, imm(rs)
'sw':    {'type': 'I', 'opcode': 0b010}, # sw rt, imm(rs)
'beq':   {'type': 'I', 'opcode': 0b011}, # beq rs, rt, label
'addi':  {'type': 'I', 'opcode': 0b100}, # addI rt, rs, imm (Case
insensitive no parser)

```

```

# Formato J

```

```

'jmp':   {'type': 'J', 'opcode': 0b111}

```

```

}

```

```

def parse_register(token):

```

```

    token = token.upper().replace(',', ' ')

```

```

    if token in REGISTERS:

```

```

        return REGISTERS[token]

```

```

    raise ValueError(f"Registrador inválido: {token}")

```

```

def parse_immediate(token, labels, current_line):

```

```

    token = token.replace(',', ' ')

```

```

    # Se for label

```

```

    if token in labels:

```

```

        return labels[token]

```

```

    # Se for número (hex ou decimal)

```

```

    try:

```

```

        if token.lower().startswith('0x'):

```

```

            return int(token, 16)

```

```

        return int(token)

```

```

    except ValueError:

```

```

        raise ValueError(f"Imediato ou Label inválido: {token}")

```

```

def assemble(input_file, output_file):

```

```

    with open(input_file, 'r') as f:

```

```

        lines = f.readlines()

```

```

    labels = {}

```

```

    instructions = []

```

```

    # --- PASSAGEM 1: Mapear Labels ---

```

```

    program_counter = 0

```

```

    clean_lines = []

```

```

    for line in lines:

```

```

        # Remove comentários e espaços

```

```

        line = line.split('#')[0].split(';')[0].strip()

```

```

        if not line:

```

```

            continue

```

```

        # Verifica se é label (termina com :)

```

```

        if line.endswith(':'):

```

```

            label_name = line[:-1]

```

```

            labels[label_name] = program_counter

```

```

        continue

# Se tem label na mesma linha da instrução (ex: "LOOP: add r1, r2,
r3")
if ':' in line:
    parts = line.split(':')
    label_name = parts[0].strip()
    labels[label_name] = program_counter
    line = parts[1].strip()

if line:
    clean_lines.append(line)
    program_counter += 1

# --- PASSAGEM 2: Gerar Código de Máquina ---
machine_code = []

for i, line in enumerate(clean_lines):
    # Normalizar sintaxe estranha como lw rt, imm(rs) -> substituir ( e )
    por espaço
    # Isso transforma "lw R1, 10(R2)" em "lw R1, 10 R2"
    line_fmt = line.replace('(', ' ').replace(')', ' ').replace(',', ' ')
    tokens = line_fmt.split()

    mnemonic = tokens[0].lower()
    if mnemonic == 'addi': mnemonic = 'addi' # Forçar lowercase

    if mnemonic not in INSTRUCTIONS:
        print(f"Erro na linha {i}: Instrução desconhecida '{mnemonic}'")
        continue

    info = INSTRUCTIONS[mnemonic]
    instr_type = info['type']
    opcode = info['opcode']

    binary = 0

    try:
        if instr_type == 'R':
            # Estrutura: opcode(3) rs(3) rt(3) rd(3) shamt(3) funct(3)
            # Assembly padrão: add rd, rs, rt
            # Assembly shift: sll rd, rs, shamt
            # Assembly not: not rd, rs

            funct = info['funct']
            rd = parse_register(tokens[1])
            rs = parse_register(tokens[2])
            rt = 0
            shamt = 0

```

```

        if mnemonic in ['sll', 'srl']:
            shamt = int(tokens[3]) & 0b111
        elif mnemonic == 'not':
            rt = 0 # não usado
        else:
            rt = parse_register(tokens[3])

        binary = (opcode << 15) | (rs << 12) | (rt << 9) | (rd << 6) |
(shamt << 3) | funct

    elif instr_type == 'I':
        # Estrutura: opcode(3) rs(3) rt(3) imm(9)
        # addI rt, rs, imm | beq rs, rt, label | lw rt, imm(rs)

        rs, rt, imm = 0, 0, 0

        if mnemonic in ['lw', 'sw']:
            # Sintaxe: lw rt, imm rs (após remover parenteses)
            rt = parse_register(tokens[1])
            imm = parse_immediate(tokens[2], labels, i)
            rs = parse_register(tokens[3])
        elif mnemonic == 'beq':
            # Sintaxe: beq rs, rt, label
            rs = parse_register(tokens[1])
            rt = parse_register(tokens[2])
            imm = parse_immediate(tokens[3], labels, i) # Endereço
absoluto

        else: # addi
            # Sintaxe: addi rt, rs, imm
            rt = parse_register(tokens[1])
            rs = parse_register(tokens[2])
            imm = parse_immediate(tokens[3], labels, i)

        # Trata imediato negativo (9 bits complemento de 2)
        imm = imm & 0x1FF

        binary = (opcode << 15) | (rs << 12) | (rt << 9) | imm

    elif instr_type == 'J':
        # Estrutura: opcode(3) address(15)
        # jmp label
        target = parse_immediate(tokens[1], labels, i)
        target = target & 0x7FFF # Máscara 15 bits
        binary = (opcode << 15) | target

    machine_code.append(binary)
    print(f"{line:<30} -> {binary:018b} (Hex: {binary:05x})")

except Exception as e:
    print(f"Erro fatal na linha {i}: {line} -> {e}")

```

```

        return

# --- Salvar Arquivo para o Digital ---
with open(output_file, 'w') as f:
    f.write("v2.0 raw\n") # Cabeçalho padrão do Digital
    for code in machine_code:
        f.write(f"{code:x} ") # Escreve em hex separado por espaço
    f.write("\n")

print(f"\nSucesso! Arquivo '{output_file}' gerado.")

if __name__ == "__main__":
    # Cria um arquivo de teste se não existir
    with open("programa.asm", "w") as f:
        f.write("""c
# Exemplo de Programa ReMember
addI R1, R0, 5    # R1 = 5
addI R2, R0, 10   # R2 = 10
LOOP:
add R3, R1, R2    # R3 = R1 + R2
sub R2, R2, R1    # R2 = R2 - 5
beq R2, R0, FIM   # Se R2 == 0, vai para FIM
jmp LOOP         # Pula para LOOP
FIM:
sw R3, 0(R0)      # Salva R3 na memória
""")

    print("Compilando 'programa.asm'...")
    assemble("programa.asm", "rom.hex")

```